Unit 5
Django Framework

1. Introduction to Django Framework

What is Django?

Django is a high-level open-source web framework written in Python that allows developers to build secure, scalable, and maintainable web applications quickly.

Django follows the Model–View–Template (MVT) architecture, which separates the application into different components to make development easier and organized.

It was developed by Adrian Holovaty and Simon Willison in 2005 while working at the Lawrence Journal-World to manage news websites efficiently.

Definition

Django is a Python-based web framework that provides built-in tools and libraries to develop dynamic web applications quickly and efficiently.

Features of Django

1. Rapid Development

Django provides many built-in modules which reduce development time.
Example:

- Authentication system
- Admin panel
- Form handling

2. Security

Django protects applications from common web attacks such as:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)

3. Object Relational Mapping (ORM)

ORM allows developers to interact with the database using **Python code instead of SQL queries**.
Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Here, the **Student model automatically creates a database table**.

PrinceXcel

4. Scalability
Django can handle large and complex applications with heavy traffic.

Example:
Large websites like **Instagram** and **Pinterest** use Django.

5. Built-in Admin Interface
Django automatically generates an **admin panel** where administrators can manage data easily.
Example:

- Add students
- Delete records
- Update information

Advantages of Django

- Fast development
- Secure framework
- Clean and maintainable code
- Built-in authentication system
- Easy database integration
- Large community support

Applications of Django

Django is used to build:

- E-commerce websites
- Social media platforms
- Educational websites
- Blogging systems
- Content management systems

Example:
An **online student management system** where teachers can manage student data.

2. Django Project Installation in Virtual Environment

What is a Virtual Environment?

A **Virtual Environment** is an isolated Python environment where project-specific packages are installed separately from the global Python installation.

Why Virtual Environment is Used?

1. Avoid package conflicts
2. Maintain different versions of libraries
3. Manage project dependencies easily

Example:
One project may use **Django 3**, while another project may use **Django 5**.

PrinceXcel

Steps to Install Django in Virtual Environment

Step 1: Install Virtual Environment

pip install virtualenv

Step 2: Create Virtual Environment

python -m venv myenv

Example:

python -m venv django_env

- Here **django_env** is the name of the environment.

Step 3: Activate Virtual Environment

For **Windows:**

jango_env\Scripts\activate

After activation, the environment name appears before the command prompt.

Example:

(django_env) C:\project>

Step 4: Install Django

pip install django

Step 5: Verify Installation

django-admin --version

Example Output: 5.0

- This confirms Django is successfully installed.

3. Phases in Django Project Creation

Developing a Django project involves several phases.

Phase 1: Environment Setup

In this phase:

- Install Python
- Install Django
- Create Virtual Environment
- Activate Virtual Environment

Example:

python -m venv myenv
pip install Django

PrinceXcel

Phase 2: Create Django Project
A Django project is created using the command: jango-admin startproject projectname
Example:

```
django-admin startproject college
```

- This command creates the project folder structure.

Phase 3: Create Application
In Django, a **project can contain multiple applications**.

Example:
A college project may contain:
- student app
- teacher app
- attendance app

Command: python manage.py startapp student

Phase 4: Define Models

Models define the structure of the database table.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    course = models.CharField(max_length=50)
    age = models.IntegerField()
```

Phase 5: Migration

Migration converts models into database tables.

Commands:

python manage.py makemigrations
python manage.py migrate

Example: This creates a **Student table in the database**.

Phase 6: Create Views
Views contain the logic of the application.
Example:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django Website")
```

PrinceXcel

Phase 7: Configure URLs

URLs connect user requests to views.

Example:

```
from django.urls import path
from . import views

urlpatterns = [
path('', views.home)
]
```

Phase 8: Create Templates

Templates are HTML files used to display web pages.

Example:

```
<html>
	<head>
		<title>Home Page</title>
	</head>
	<body>

		<h1>Welcome to Django</h1>

	</body>
</html>
```

4. Create a Django Project

Definition

A **Django project** is a collection of configuration files and applications used to build a complete web application.

Creation of App in Django

What is a Django App?

In Django, an **application (app)** is a module that performs a specific function within a project. A **Django project can contain multiple apps**, and each app handles a particular feature of the website.

Example

Consider a **College Management System** project.

It may contain different apps such as:

- **Student App** – manages student information
- **Teacher App** – manages teacher data
- **Attendance App** – manages attendance records

Each app works independently but is part of the same project.

PrinceXcel

Steps to Create an App in Django

Step 1: Navigate to the Project Directory

Open the terminal or command prompt and go to your project folder.

Example:

```
cd myproject
```

Step 2: Create an App

Use the following command:

```
python manage.py startapp appname
```

Example:

python manage.py startapp student

- Here **student** is the name of the application.

Step 3: Register the App in settings.py

After creating the app, it must be **registered in the project settings**.

Open **settings.py** and add the app name in **INSTALLED_APPS**.

Example:

```
INSTALLED_APPS = [
 'django.contrib.admin',
  'django.contrib.auth',
  'student',
]
```

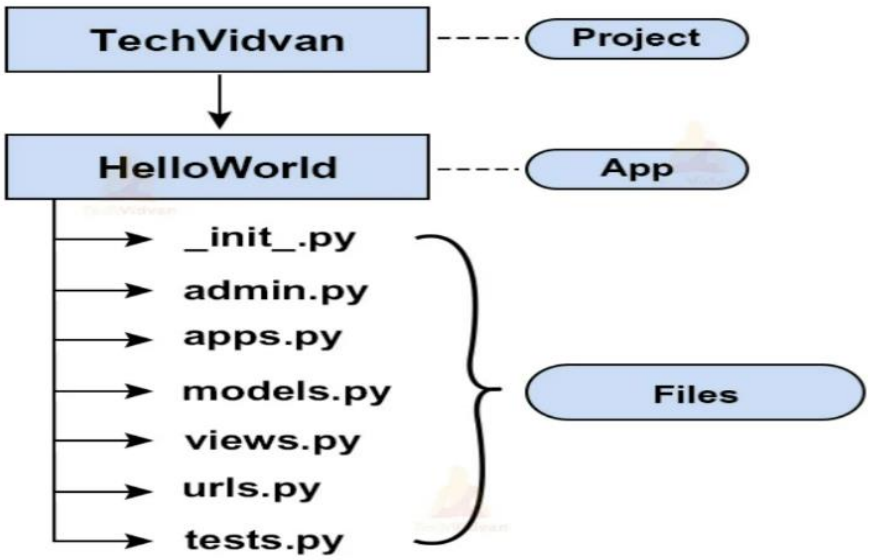
- This allows Django to recognize and use the app.

5.Structure of a Django App

After creating the app, Django automatically generates several files.

Example structure:

PrinceXcel



Explanation of App Files

1. models.py

This file is used to create **database models (tables)**.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

This creates a **Student table in the database**.

2. views.py

Views contain the **application logic** and control what the user sees.

Example:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Student App")
```

3.**admin.py**

This file is used to **register models in the admin panel**.

Example:

```
from django.contrib import admin
from .models import Student

    admin.site.register(Student)
```

PrinceXcer

4. apps.py

Contains configuration settings for the application.

Example:

```
from django.apps import AppConfig

class StudentConfig(AppConfig):
    name = 'student'
```

5. migrations folder

This folder stores **database migration files**. Migration files track changes made to models.

Example commands:

```
python manage.py makemigrations
python manage.py migrate
```

6. tests.py

Used to write **test cases** for the application.

Example:

```
from django.test import TestCase
```

6. Working with Django Admin Console

What is Django Admin Console?

The **Admin Console** is a built-in feature of Django that provides a **web-based interface for managing database records**.

It allows administrators to:

- Add data
- Update records
- Delete records
- Manage users

without writing additional code.

Steps to Use the Admin Console

Step 1: Create a Superuser

To access the admin console, create an admin account.

Command:

```
python manage.py createsuperuser
```

PrinceXcel

Example Input:

```
Username: admin
Email: admin@gmail.com
Password: *****
```

Step 2: Run the Server

Start the Django development server.

```
python manage.py runserver
```

Step 3: Open the Admin Panel

Open the browser and go to:

```
http://127.0.0.1:8000/admin
```

Login using the **superuser username and password**.

Registering Models in Admin Panel

To display models in the admin console, they must be **registered in admin.py**.

Example:

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

After registering, the **Student model will appear in the admin dashboard**.

Example: Working with Admin Console

Step 1: Create Model

```
from django.db import models
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
```

Step 2: Register Model in Admin

```
from django.contrib import admin
from .models import Student

    admin.site.register(Student)
```

Step 3: Run Migration

```
python manage.py makemigrations
python manage.py migrate
```

PrinceXcel

Step 4: Add Data Through Admin Panel

Steps:

1. Open **admin panel**
2. Click **Student**
3. Click **Add Student**
4. Enter data
5. Click **Save**

Advantages of Django Admin Console

1. Easy data management
2. No need to write extra code
3. Automatically generated interface
4. Secure authentication system
5. Quick data updates

7. Creating Views in Django

What is a View?

In Django, a **View** is a Python function or class that receives a web request and returns a web response. Views contain the **business logic of the application** and determine what data will be displayed to the user.

Definition

A **View** is a Python function that takes a request object as an argument and returns a response such as an HTML page, JSON data, or a simple message.

Example of a View

Open **views.py**

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django Website")
```

Explanation

- **request** → HTTP request from user
- **HttpResponse()** → sends response to browser

Types of Views

1. Function-Based Views

These are simple Python functions.

Example:

```
def about(request):
    return HttpResponse("About Page")
```

2. Class-Based Views

These use Python classes to create views.

PrinceXcel

Example:

```
from django.views import View
from django.http import HttpResponse

class HomeView(View):
def get(self, request):
        return HttpResponse("Welcome")
```

8.URL Mapping in Django

What is URL Mapping?

URL Mapping connects a **URL address with a view function**. When a user enters a URL in the browser, Django checks the **urls.py** file and calls the corresponding view.

Example

Open **urls.py**

```
from django.urls import path
from . import views

urlpatterns = [
 path('', views.home),
 path('about/', views.about),
]
```

Explanation

URL	View
/	home view
/about	about view

Example URL:

http://127.0.0.1:8000/about

This will call the **about() view**.

Template System in Django

What is a Template?

A **Template** is an HTML file used to display data dynamically. Django templates allow developers to insert **dynamic content into HTML pages**.

PrinceXcel

Template Folder Structure

Create a folder named:

```
templates
```

Example structure:

```
project/
    templates/
        home.html
```

Example Template

home.html

```
<html>
<head>
    <title>Home Page</title>
</head>

<body>

    <h1>Welcome to Django Website</h1>
</body>
</html>
```

Rendering Template in View

Open **views.py**

```
from django.shortcuts import render

def home(request):
    return render(request, "home.html")
```

Passing Data to Template

Example:

```
def home(request):
    data = {"name": "Rahul"}
    return render(request, “home.html”, data)
```

Template:

```
<h1>Welcome {{name}}</h1>
```

Output:

```
Welcome Rahul
```

9.Working with Models

What is a Model?

A **Model** represents a **database table** in Django.

Models define the **structure of data stored in the database**.

PrinceXcel

Example Model

Open **models.py**

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    course = models.CharField(max_length=50)
```

Model Fields

Field	Description
CharField	Text field
IntegerField	Numeric field
DateField	Date value
EmailField	Email address

Migration Commands

After creating models run:

```
python manage.py makemigrations
python manage.py migrate
```

These commands create tables in the database.

10. Form Processing in Django

What is Form Processing?

Form processing is used to **collect data from users through HTML forms and store it in the database**.

Example:

- Registration form
- Login form
- Feedback form

Example HTML Form

```
<form method="POST">

    <input type="text" name="name">

    <input type="submit">

</form>
```

PrinceXcel

Handling Form in Views

```
from django.shortcuts import render

def form_view(request):

if request.method == "POST":
  name = request.POST.get("name")
  print(name)

        return render(request, "form.html")
```

Explanation

- **request.POST** → retrieves form data
- **POST method** → used to send data securely

11. Static and Media Files

Static Files

Static files are **files that do not change**.

Examples:

- CSS
- JavaScript
- Images

Media Files

Media files are **user uploaded files**.

Examples:

- profile pictures
- documents
- videos

Example model for media file:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    photo = models.ImageField(upload_to='images/')
```

Uploaded images are stored in the **media folder**.

12. Django App Deployment

What is Deployment?

Deployment is the process of **making a Django application available on the internet** so users can access it.

Example:

Instead of running locally:

http://127.0.0.1:8000

The website becomes accessible online.

PrinceXcel

Steps for Deployment

1. Set **DEBUG = False** in settings.py
2. Configure **ALLOWED_HOSTS**
3. Collect static files

Example:

python manage.py collectstatic

4. Use a web server like:

- **Nginx**
- **Gunicorn**

5. Host the project on a cloud server.

Popular Platforms for Deployment

Examples:

- **Amazon Web Services**
- **Heroku**
- **DigitalOcean**

*****

PrinceXcel